Song Sorters

Course: COP3530 | Project: Project 2 | Group: Song Sorters

Members: Alwin Paul (GitHub: alwinp2006-stack), Rafael Hitoshi Teixeira Oura, and Ricardo Chiacchio Ribeiro

Repository: https://github.com/Rafatoshi/Song-Sorter-COP3530

Demonstration video: [Video link]

1. Introduction

The goal of this project is to sort a list of songs by view count and compare the performance of two sorting algorithms: Merge Sort and Quick Sort. Each song contains a title and a number of views. We generate two data sets containing 500 and 10,000 songs.

The project is a menu-driven C++ console application. It generates song lists, sorts them with the two custom algorithms, displays the top songs, and compares runtimes. We also use C++ `std::sort` as a reference for comparison.

2. Design and Implementation

Data Structure and Song Generation

A `Song` struct stores a song's `title` (string) and `viewCount` (integer). Songs are stored in a vector, which supports a variable number of records and indexed access.

The `generateSongs()` function creates the requested number of songs. Titles are unique (for example, “Song 1”), and view counts are random values from 0 to 1 billion. A fixed random seed makes the generated data repeatable for testing.

Merge Sort

Merge Sort uses divide and conquer. It repeatedly splits the list until each part contains one element, then merges the parts in descending order of view count. During merging, a temporary vector stores the combined result. When two songs have equal view counts, the left element is selected first, so Merge Sort is stable.

Complexity: O(n log n) time in the best, average, and worst cases, with O(n) auxiliary space.

2. Design and Implementation (continued)

Quick Sort

Quick Sort also uses divide and conquer. The pivot is selected using the median-of-three method, which considers the first, middle, and last elements. Hoare partitioning moves songs with higher view counts toward the front and lower view counts toward the end.

The implementation recursively sorts the smaller partition and handles the larger partition in a loop to limit recursion depth. Quick Sort is not stable, so equal-view songs are not guaranteed to retain their original order.

Complexity: O(n log n) average time and O(n^2) worst-case time when partitions are repeatedly uneven. Recursing on the smaller partition keeps the recursion stack at O(log n).

Benchmarking

Merge Sort and Quick Sort are implemented from scratch. The C++ `std::sort` function is used only as a reference. Each algorithm receives its own copy of the same original, unsorted data set. The timer starts immediately before sorting and stops immediately afterward, so vector copying is not included in the measured sort time.

After sorting, the program checks that songs are in descending order and that all original song records remain present. The algorithms are compared on both data-set sizes.

3. Testing and Runtime Results

We tested empty lists, one-song lists, already sorted lists, unsorted lists, duplicate view counts, larger lists, and whether all songs were retained. The benchmark also compares the custom sorts with the reference sort and checks the original song titles and view counts.

Catch2 test results: [Number of tests passed] passed; [Number of tests failed] failed.

3. Testing and Runtime Results (continued)

Runtime for 500 Songs

- Merge Sort: [Time taken] milliseconds.
- Quick Sort: [Time taken] milliseconds.
- std::sort: [Time taken] milliseconds.

Runtime for 10,000 Songs

- Merge Sort: [Time taken] milliseconds.
- Quick Sort: [Time taken] milliseconds.
- std::sort: [Time taken] milliseconds.

The measured times show which algorithm ran fastest for each data-set size. Merge Sort has O(n log n) time in all cases. Quick Sort averages O(n log n), but its performance depends on pivot quality and partition balance. Median-of-three pivot selection reduces the likelihood of highly uneven partitions. The completed benchmark values will allow us to compare the theoretical analysis with observed performance.

4. Time Complexity of Key Functions

- **Generate Songs:** O(n) time and O(n) space to create and store n songs.
- **Merge Sort:** O(n log n) time in all cases; O(n) auxiliary space for merging.
- **Quick Sort:** O(n log n) average time, $O(n^2)$ worst-case time, and O(log n) recursion-stack space with smaller-partition recursion.
- **Benchmark:** Sorting time depends on the selected algorithm. Validation adds a separate pass over the data.
- **Display Top 10:** The program copies and sorts the vector using a custom sort, giving O(n log n) overall time.

5. Modifications and Team Responsibilities

The original proposal described a website. We instead built a menu-driven console application to keep the focus on implementing and comparing the sorting algorithms. During development, we corrected issues in the Merge Sort merge process and improved benchmark validation.

We also confirmed that both algorithms sort from highest to lowest view count, handled duplicate view counts according to each algorithm's stability, and made benchmarking fair by giving each algorithm a separate copy of the original unsorted vector.

- **Alwin Paul:** Generate song data and support the two data-set sizes.
- **Rafael Hitoshi Teixeira Oura:** Implement Merge Sort and tests.
- **Ricardo Chiacchio Ribeiro:** Implement Quick Sort and tests.
- **All members:** Benchmark system, results analysis, report writing, and demonstration video.

6. Reflection

This project gave us practical experience using two sorting algorithms to solve the same problem. Both use divide and conquer, but they differ in implementation, memory use, stability, and worst-case performance. Merge Sort has consistent O(n log n) time but requires extra memory for merging. Quick Sort generally performs well and uses a smaller recursion stack, but its worst-case time is $O(n^2)$.

Testing was important because an algorithm that works on a small list might fail with larger data sets or duplicate values. Comparing the custom algorithms with a reference sort and checking that all song records remained present helped us validate the results. We also gained experience with C++ structs, vectors, testing, and runtime measurement.

6. Reflection (continued)

Individual learning reflections: [Add one sentence describing what Alwin learned.] [Add one sentence describing what Rafael learned.] [Add one sentence describing what Ricardo learned.]

7. References

- University of Florida. COP3530 Sorting Algorithms Lecture.
- cppreference.com. “std::sort.” https://en.cppreference.com/w/cpp/algorithm/sort
- cppreference.com. “std::chrono.” https://en.cppreference.com/w/cpp/chrono

Project repository: https://github.com/Rafatoshi/Song-Sorter-COP3530

Demonstration video: [Video link]